

Министерство образования Республики Беларусь

Учреждение высшего образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе №2
на тему

РАБОТА СО СПИСКАМИ И ФУНКЦИЯМИ

Студенты группы 150503:

Шарай П.Ю.
Соломко В. Ю.

Проверила:

Герман Ю. О.

Минск 2023

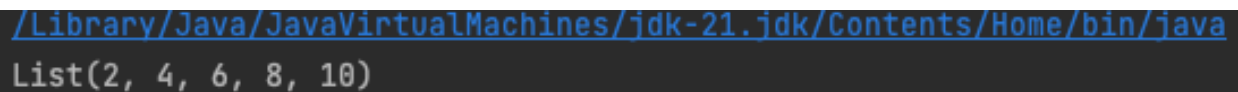
Цель: изучить технику работы со списками и функциями в Scala.

1. Краткие теоретические сведения:

Примеры функций для работы со списками в Scala из методички:

В первом примере выполнения перебор всех элементов с помощью `map` и умножение каждого элемента на 2. Кода данной программы приведен ниже, а выполнение данного кода на рисунке 1.

```
object Main22 {  
  
    def double(x: Int): Int = x * 2  
  
    def main(args: Array[String]): Unit = {  
        val myList = List(1, 2, 3, 4, 5)  
        val doubledList = myList.map(double)  
        println(doubledList)  
    }  
}
```

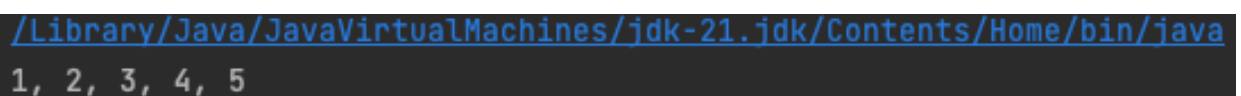


```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java  
List(2, 4, 6, 8, 10)
```

рисунок 1 - пример выполнения примера

Также продемонстрирован по элементный вывод List. Пример выполнения данного кода приведен на рисунке 2.

```
object Main22 {  
  
    def double(x: Int): Int = x * 2  
  
    def main(args: Array[String]): Unit = {  
        val myList = List(1, 2, 3, 4, 5)  
        val doubledList = myList.map(double)  
        println(myList.mkString(", "))  
    }  
}
```



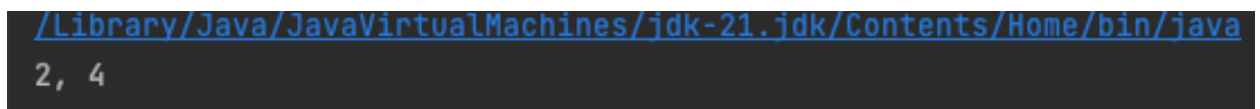
```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java  
1, 2, 3, 4, 5
```

рисунок 2 - пример выполнения примера

Далее у нас рассматривается метод `filter` - эта функция отбирает элементы списка, удовлетворяющие заданному предикату.

```
object Main22 {  
  
  def isEven(x: Int): Boolean = x % 2 == 0  
  
  def main(args: Array[String]): Unit = {  
    val myList = List(1, 2, 3, 4, 5)  
  
    val filteredList = myList.filter(isEven)  
    println(filteredList.mkString(", "))  
  }  
}
```

Пример выполнения данного кода приведен на рисунке 3.



```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java  
2, 4
```

рисунок 3 - пример выполнения примера

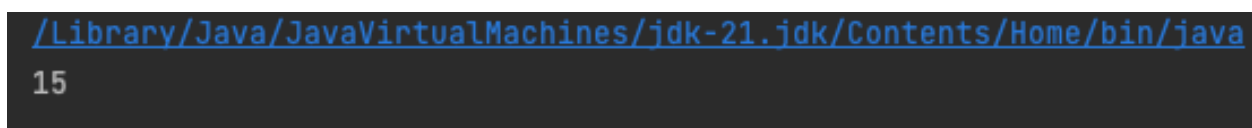
`FoldLeft` – эта функция последовательно применяется к элементам списка слева направо, накапливая результат. Сумму элементов списка можно найти таким образом

```
object Main22 {  
  
  def main(args: Array[String]): Unit = {  
    val myList = List(1, 2, 3, 4, 5)  
    val sum = myList.foldLeft(0)((ac_c, x) => ac_c + x)  
    println(sum) // Output: 15  
  }  
}
```

Здесь переменная `ac_c` играет роль аккумулятора. Первоначально ей присваивается значение 0:

```
myList.foldLeft(0)
```

Пример выполнения данного кода приведен на рисунке 4.



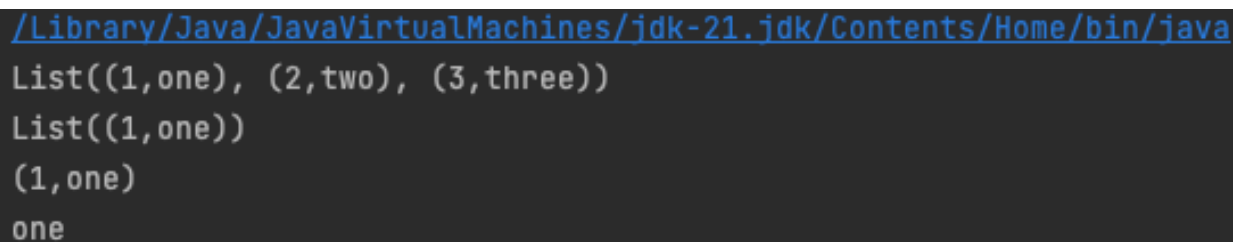
```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java  
15
```

рисунок 4 - пример выполнения примера

Zip - эта функция объединяет два списка на примере словаря (dictionary) – ключ-значение.

```
val a = List(1, 2, 3)
val b = List("one", "two", "three")
val zipped = a.zip(b)
println(zipped)
}
```

Пример выполнения данного кода приведен на рисунке 5.



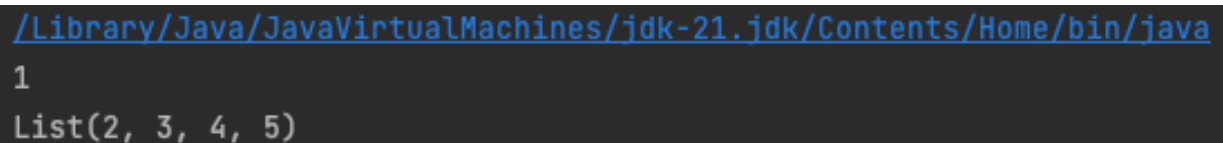
```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java
List((1,one), (2,two), (3,three))
List((1,one))
(1,one)
one
```

рисунок 5 - пример выполнения примера

Head and tail - эти функции возвращают голову и хвост списка соответственно.

```
val numbers = List(1, 2, 3, 4, 5)
val first = numbers.head
val rest = numbers.tail
```

Пример выполнения данного кода приведен на рисунке 6.



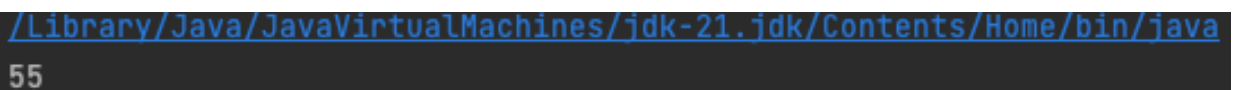
```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java
1
List(2, 3, 4, 5)
```

рисунок 6 - пример выполнения примера

Сумма квадратов элементов списка :

```
object Main22 {  
  def sumList(lst: List[Int]): Int = {  
    if (lst.isEmpty) 0  
    else lst.head*lst.head + sumList(lst.tail)  
  }  
  
  def main(args: Array[String]): Unit = {  
    val myList = List(1, 2, 3, 4, 5)  
    val sumw = sumList(myList)  
    println(sumw)  
  }  
}
```

Пример выполнения данного кода приведен на рисунке 7.



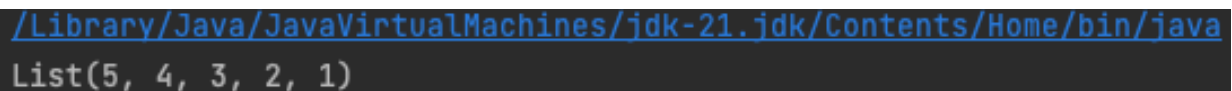
```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java  
55
```

рисунок 7 - пример выполнения примера

Reverse – эта функция возвращает список в обратном порядке.

```
val numbers = List(1, 2, 3, 4, 5)  
val reversed = numbers.reverse
```

Пример выполнения данного кода приведен на рисунке 8.



```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java  
List(5, 4, 3, 2, 1)
```

рисунок 8 - пример выполнения примера

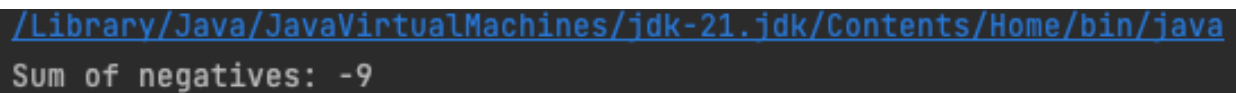
2. Ход работы:

Вариант 2.

1. Написать функцию для подсчета суммы отрицательных элементов списка. Список задать самостоятельно.

Код первой программы и результаты выполнения:

```
def sumOfNegatives(list: List[Int]): Int = {  
  if (list.isEmpty) {  
    0  
  } else if (list.head < 0) {  
    list.head + sumOfNegatives(list.tail)  
  } else {  
    sumOfNegatives(list.tail)  
  }  
}
```



```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java  
Sum of negatives: -9
```

рисунок 9 - пример выполнения первой программы

2. Написать функцию для подсчета суммы последних трех элементов списка из 10 элементов. Список задать самостоятельно.

Код второй программы и результаты выполнения:

```
def sumOfLastThree(list: List[Int]): Int = {  
  if (list.isEmpty) {  
    0  
  } else if (list.length <= 3) {  
    list.sum  
  } else {  
    sumOfLastThree(list.tail)  
  }  
}
```



```
Sum of last three: 27
```

рисунок 10 - пример выполнения второй программы

3. Написать функцию для отыскания индексов всех максимальных элементов списка. Список задать самостоятельно.

Код третьей программы и результаты выполнения:

```
def indexesOfMax(list: List[Int]): List[Int] = {  
  def loop(list: List[Int], max: Int, index: Int, indices: List[Int]):  
  List[Int] = {  
    list match {
```

```

        case Nil => indices
        case head :: tail if head > max => loop(tail, head, index + 1,
List(index))
        case head :: tail if head == max => loop(tail, max, index + 1,
indices :+ index)
        case _ :: tail => loop(tail, max, index + 1, indices)
    }
}

list match {
  case Nil => Nil
  case head :: tail => loop(tail, head, 1, List(0))
}
}

```

Indexes of max: List(3)

рисунок 11 - пример выполнения третьей программы

4. Написать функцию для проверки того, что список не упорядочен не по убывания, не по возрастанию. Список задать самостоятельно.

Код четвертой программы и результаты выполнения:

```

def isUnordered(list: List[Int]): Boolean = {
  def isUnorderedRec(list: List[Int], ascending: Boolean): Boolean = {
    list match {
      case Nil => true
      case _ :: Nil => true
      case head1 :: head2 :: tail =>
        if ((head1 < head2 && ascending) || (head1 > head2 && !
ascending))
          isUnorderedRec(tail, ascending)
        else
          false
    }
  }
}

isUnorderedRec(list, ascending = true) && isUnorderedRec(list,
ascending = false)
}

```

Is unordered: false

рисунок 12 - пример выполнения четвертой программы

5. Написать функцию для проверки наличия трех одинаковых элементов в списке. Список задать самостоятельно. Функция возвращает значение такого элемента.

Код пятой программы и результаты выполнения:

```
def hasThreeEqual(list: List[Int]): Option[Int] = {  
  def countOccurrences(element: Int, lst: List[Int]): Int = {  
    lst match {  
      case Nil => 0  
      case head :: tail if head == element => 1 +  
countOccurrences(element, tail)  
      case _ :: tail => countOccurrences(element, tail)  
    }  
  }  
  
  list.find { element => countOccurrences(element, list) >= 3 }  
}
```



```
Has three equal: Some(3)  
Has three equal: None
```

рисунок 13 - пример выполнения четвертой программы

3. Вывод:

В ходе выполнения данной лабораторной работы нами были проанализированы и изучены техники работы со списками и функциями в Scala.